Communication-Avoiding Krylov Exponential Integration for Multi-GPU Option Pricing

Kevin Knights
Supervisor: Prof. Kirk M. Soodhalter

MSc. High Performance Computing
School of Mathematics, Trinity College Dublin

August 31, 2026

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

A trading desk reprices the same multi-asset PDE across a whole book, so the cost of *one* solve sets valuation throughput. That solve is dominated by **moving data**, not arithmetic, which is what *communication-avoiding* methods target.

The question is not *how* to avoid communication, but **when it is worth doing.**

1. **The problem and the method**
   the operator, and one exponential action per step
2. **Where the time goes**
   profiling, the two CA mechanisms, and a null result
3. ★ **The regime map**
   two a-priori coordinates that predict the opportunity
4. ★ **Does it pay?**
   a distributed CA integrator on four GPUs

Method in one line

Predict the opportunity *before* building the kernel, then measure whether the prediction holds.

★ marks my own contribution.

1. The problem and the method
2. Where the time goes
3. The regime map
4. Does it pay?

The problem: one operator, solved over and over

Three-asset Black–Scholes in log prices, backward in $\tau = T - t$, for a **basket call** and a **rainbow min-call**:

$$\frac{\partial u}{\partial \tau} = \underbrace{\frac{1}{2}\sum_{d,d'} \rho_{dd'}\sigma_d\sigma_{d'} \frac{\partial^2 u}{\partial x_d \partial x_{d'}}}_{\text{diffusion+correlation}} + \underbrace{\sum_{d} \left(r - \tfrac{1}{2}\sigma_d^2\right)\frac{\partial u}{\partial x_d}}_{\text{convection}} \underbrace{- ru}_{\text{reaction}} + b(\tau)$$

Second-order central differences on a uniform grid, $N = n^3$ unknowns:

- A carries **19 nonzeros per row** and $O(N_1 N_2)$ bandwidth, which rules out banded direct solvers at scale.
- Convection is skew-symmetric, so A is **non-symmetric and non-normal**.
- Stiffness grows as $\|A\| \sim O(n^2)$.

Production resolution is $n = 61$: **226,981 unknowns**, 4.23 M nonzeros.

The method: one exponential action per time step

Discretizing space only leaves time continuous: the PDE becomes a large linear system of ODEs, one equation per grid point.

$$\frac{d\mathsf{u}}{d\tau} = \mathsf{A}\mathsf{u} + \mathsf{B}\mathsf{s}(\tau), \qquad \mathsf{u}(0) = \mathsf{u}_0 \quad \text{(the payoff)}.$$

The basket's boundary forcing is polynomial, $\mathsf{s}' = \mathsf{K}\mathsf{s}$ with K nilpotent, so it is absorbed *exactly* into an autonomous augmented system:

$$\widetilde{\mathsf{A}} = \begin{bmatrix} \mathsf{A} & \mathsf{B} \\ 0 & \mathsf{K} \end{bmatrix}, \qquad \boxed{\widetilde{\mathsf{v}} = \begin{bmatrix} \mathsf{u}_k \\ \mathsf{s}(\tau_k) \end{bmatrix}, \qquad M = N + 3}$$

One time step is then *one matrix-exponential action*, approximated in the Krylov subspace $\mathcal{K}_m(\mathsf{G}, \widetilde{\mathsf{v}})$ with $\mathsf{G} = h\widetilde{\mathsf{A}}$:

$$\mathsf{u}_{k+1} = e^{\mathsf{G}}\widetilde{\mathsf{v}} \;\approx\; \beta\, \mathsf{V}_m\, e^{\mathsf{H}_m} \mathsf{e}_1, \qquad \beta = \|\widetilde{\mathsf{v}}\|.$$

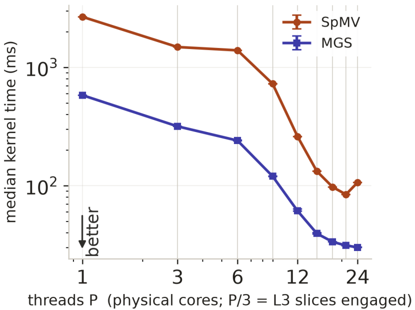
Arnoldi on G builds V_m and $\overline{\mathsf{H}}_m$; m grows adaptively until the endpoint defect $\eta_m < 10^{-8}$.

Profiling: two kernels, two different ceilings

Kernel	n=31	n=61
SpMV	83.5%	78.0%
MGS	11.4%	16.6%
dense $\exp(H_m)$	0.40%	0.23%
mean m	7.60	8.91

share of KSM-EI solve time

Grids follow Niesen & Wright's option-pricing study, so the semi-discrete ODE error stays comparable with theirs.



Strong scaling at n=61: fixed work, more cores.

SpMV plateaus until the working set fits the engaged cache. Modified Gram–Schmidt (MGS) keeps improving, but every inner product it performs is a global reduction, and its share grows with m.

Communication avoidance: two mechanisms

The obstacle. Arnoldi iteration j needs v_j before it can start: one SpMV and one global reduction per step, strictly in sequence.

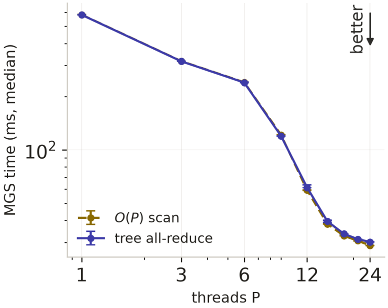
1. One deep exchange, not $q-1$.

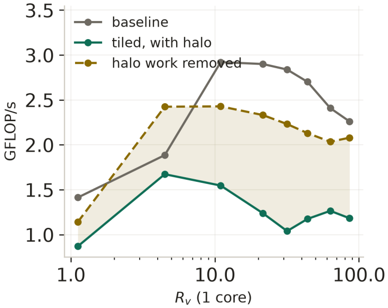
2. One block, not m vectors. Orthogonalize a whole Krylov block at once with **CholeskyQR2**: form the Gram matrix $Z^{\mathsf{T}}Z$, Cholesky-factor it, repeat once for stability.

$$Z_q = [\mathsf{v},\ A\mathsf{v},\ \ldots,\ A^{q-1}\mathsf{v}] \qquad \underbrace{1 + \frac{m(m+3)}{2}}_{\text{MGS}} \longrightarrow \underbrace{1 + 2\lceil m/q \rceil}_{\text{one block}}$$

Messages fall from $q-1$ to one and reductions by **11–16**$\times$ at the measured $m = 8$–11, $q = 8$; the price is recomputing the ghost region.

Both mechanisms built and timed on *puffin*, a 24-core Threadripper 3960X on a single NUMA socket.





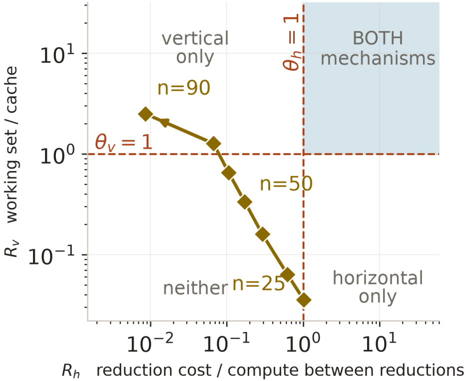
Horizontal: a 1.7× faster all-reduce leaves MGS within 5%.

Vertical (L2): the gap is the halo. The L3 sweep agrees.

11–16× fewer reductions, 4–8× less modeled traffic.
The saving is real; the speedup is not. So when *does* it pay?

The regime map: predict first, then measure

$$R_v = \frac{\text{Arnoldi working set}}{\text{last-level cache}} \qquad R_h = \frac{\text{reduction latency}}{\text{compute between reductions}}$$



Eight production grids, $n = 25 \to 90$, real basket operator, puffin's calibrated constants.

On one socket $R_v R_h$ stays near 0.04 for every P, so the corner is not merely unreached here: it is unreachable on this topology.

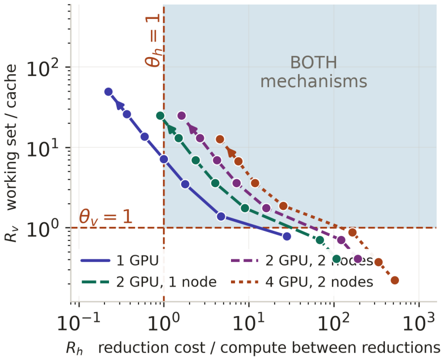
R_v, **from the problem.** Working-set bytes implied by N, the sparsity pattern and the measured Krylov dimension m, over the last-level cache actually available to the team:

$$R_v = \frac{\mathrm{bytes}(N,\ \mathrm{nnz},\ m)}{C_{\mathrm{LLC}}(P)}.$$

R_h, **from a calibration instrument.** An isolated scalar all-reduce with no application work, fitted over the tree levels it crosses:

$$R_h = \frac{t_{\mathrm{red}}(P)}{W_{\mathrm{local}}}, \qquad t_{\mathrm{red}}(P) = t_{\mathrm{intra}} L_{\mathrm{in}} + t_{\mathrm{cross}} \min(L_{\mathrm{x}}, L^{\star}).$$

Raising the cost of a reduction is what moves an operator right, and that is a property of topology.
Puffin tops out at $t_{\mathrm{red}} = 1.70\,\mu\mathrm{s}$, and the corner opens at about 25 t_{red}.



Same definitions, recalibrated machine constants. Synge is two nodes with two Tesla V100s each.

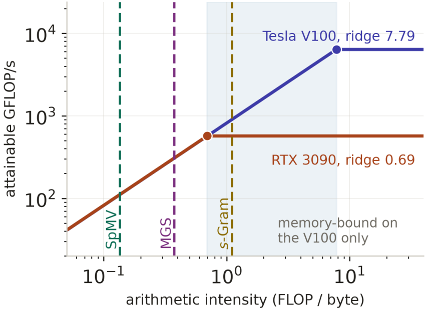
A GPU crosses **five** reduction levels where the CPU crosses two, and every added participant raises t_{red}, moving the whole trajectory **right**.

At n=61, m=12, q=8 the coordinates are $R_v = 7.2$ and $R_h = 1.7$: inside the corner on both cards.

Each line sweeps the same grids: n=25 at the lower right to n=90 at the upper left.

1. The problem and the method
2. Where the time goes
3. The regime map
4. Does it pay?

Reaching the corner is not enough: the device's roofline



Nsight Compute, *s*-Gram kernel

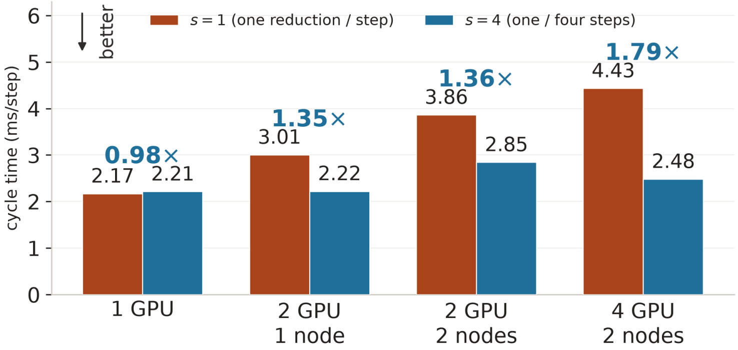
% of roof	V100	3090
DRAM	**86%**	38%
compute	16%	**94%**
verdict	memory	compute

The memory-bound check

A precondition, before the map is read: compare each kernel's arithmetic intensity (AI, FLOP per byte) with the FP64 ridge, peak FP64 over achieved bandwidth (BW).

$$AI_{\text{kernel}} < \underbrace{\frac{\text{peak FP64}}{BW_{\text{achieved}}}}_{\text{FP64 ridge}} \implies \text{memory-bound, and on the map.}$$

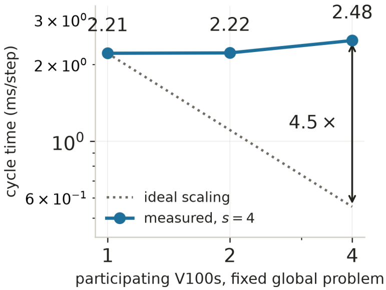
Does communication avoidance pay?



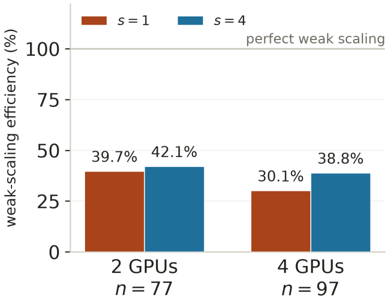
Basket, production grid n=61, 100 steps, *exact-depth* halo: both widths do equal numerical work.

Parity on one GPU, and **1.78–1.80**× on four V100s across two nodes.
Plane-streamed matrix powers add **1.21–1.32**× on a single device.

The honest limit: CA helps, distribution still costs



Strong: fixed problem, more GPUs.
0.89×, about 22% efficiency.



Weak: ≈227,000 rows per GPU held fixed.

At n=61 each of four slabs owns only ≈57,000 rows: too little to saturate a V100. Four GPUs earn their place **above** $n = 337$, where the problem stops fitting on one card.

Communication avoidance pays only when the **operator**, the **kernel** and the **machine** all line up.

What the work establishes

- Two a-priori coordinates that say *which* mechanism is worth measuring, without timing the solver.
- One system where the corner is unreachable and one where more participants reach it, on the same definitions.
- A distributed CA integrator, **1.78–1.80**$\times$ faster on four V100s, validated against independent references and **5.8–6.4**$\times$ faster than smoothed ADI at equal declared accuracy.

Future work

- A **tile-aware** R_v predicting read redundancy from tile dimensions, stencil radius and streamed height.
- **Mixed precision** recurrence with higher-precision Gram accumulation. Not free: a lower-precision basis worsens the very conditioning the guard tracks, so it must be gated on measured orthogonality.
- **H100-class devices and faster links**: does the crossover transfer beyond Synge?

Thank you

This work stands on a great deal of help.

My thanks to my supervisor, to the staff and students of the MSc in High-Performance Computing, to everyone who kept the clusters running, and to my family, for the teaching, the patience and the support that made it possible.

"If I have seen further it is by standing on the shoulders of Giants."

Isaac Newton, letter to Robert Hooke, 5 February 1675

Questions?

Code: `github.com/kevinknights29/caksm`

Thesis: `.../caksm/docs/thesis/main.pdf`

Appendix

Supporting detail, held in reserve.

Are the prices still right, and how does the solver compare with an established method? **KSM-EI** is the Krylov exponential integrator, **ADI-HV-S** the smoothed Hundsdorfer–Verwer ADI scheme.

Equal declared accuracy, one thread, n=61

Payoff	Method	Steps	Median (s)	
Basket	ADI-HV-S	50	3.090	
Basket	KSM-EI	1	0.533	**5.8**×
Rainbow	ADI-HV-S	50	2.936	
Rainbow	KSM-EI	1	0.460	**6.4**×

Where the price error actually is

Source of Δ	Basket	Rainbow
reference	6.8×10^{-6}	$<10^{-13}$
domain	1.0×10^{-4}	4.5×10^{-6}
spatial	2.9×10^{-3}	3.6×10^{-2}
ADI temporal	2.2×10^{-6}	2.4×10^{-5}
KSM temporal	5.3×10^{-15}	3.6×10^{-15}

Both solvers refine at second order against independent references.
Basket Δ errors are 0.011–0.020%. The ADI comparison is **Dang, Christara & Jackson (2010)**, my last seminar paper.

A monomial block $[\mathsf{v}, \mathsf{Av}, \ldots, \mathsf{A}^{q-1}\mathsf{v}]$ loses conditioning fast, so CholeskyQR2 needs a guard.

What the implementation does. It narrows the block whenever $\kappa_2(\mathsf{Z}) > u^{-1/2} \approx 9.5 \times 10^7$.

What theory covers. The dimension-dependent sufficient condition $\delta = 8\kappa_2(\mathsf{Z})\sqrt{Mqu + q(q+1)u} \leq 1$ limits κ_2 to $\approx 1.25 \times 10^4$ at $M = 226{,}984$, $q = 4$. The measured block reaches 6.0×10^6, giving $\delta \approx 482$: the theorem is *inconclusive*, not violated.

Acceptance therefore rests on the guard, successful factorization, measured orthogonality, and referee agreement.

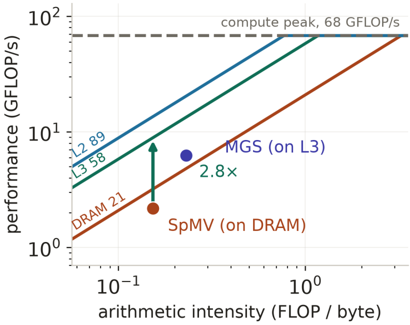
Orthogonality loss, n=61, m=8, q=4

Method	$\lVert \mathsf{I} - \mathsf{Q}^\mathsf{T}\mathsf{Q} \rVert$	**Time**
CholeskyQR2	4.0×10^{-14}	$1.00\times$
TSQR	3.5×10^{-14}	$3.06\times$
Block GS, reorth.	6.5×10^{-14}	$1.24\times$

Width admitted by the guard

Basis	n=31	n=61
Monomial	**4**	**4**
Newton (Leja)	3	3
Chebyshev	4	3

Appendix: the cache-aware roofline on puffin



One core, n=61.

Kernel	AI	Tier
SpMV	0.15	DRAM
MGS	0.23	L3
dense $\exp(\mathrm{H}_m)$	1.76	L2

arithmetic intensity, FLOP per byte

$$\text{gain} = \frac{\min(BW_{\text{fast}} \cdot AI,\ \text{peak})}{\min(BW_{\text{slow}} \cdot AI,\ \text{peak})} = \frac{58.2}{20.9} = 2.8\times \quad \text{at } AI = 0.15.$$

Appendix: where a GPU differs from a CPU

CPU

- Cache **grows with the team**: each engaged core complex brings its own L3 slice.
- R_v scales with the engaged core count P, so the two axes are coupled and $R_v R_h$ is roughly P-independent.
- Reductions cross **2** levels.

GPU

- Cache is **fixed**: engaging more SMs adds none.
- R_v stops depending on how many SMs are engaged, so **grid size alone moves it.**
- Reductions cross **5** levels, one of them a kernel launch. Coalescing also makes the access pattern far more sensitive.

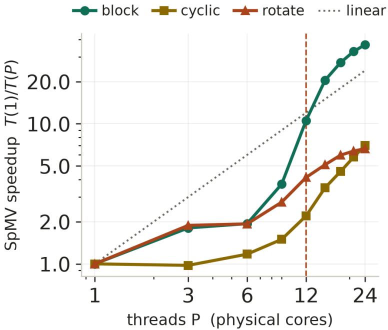
Appendix: the reduction ladder and $t^{\star}_{\mathrm{red}}$

$$R_v R_h = g(m)\,R(m)\,\Lambda, \qquad \Lambda = \frac{t_{\mathrm{red}} \cdot BW}{C} \qquad \Longrightarrow \qquad t^{\star}_{\mathrm{red}} = 0.69\,\mu s$$

priced at the production point n=61, m=17.

Reduction rung	t_{red} **(μs)**	**Corner**
warp shuffle	0.42	closed
thread block	0.79	**open**
grid, 2nd launch	5.54	**open**
device, PCIe	11.46	**open**
node, fabric	22.09	**open**

Puffin, for comparison: $t_{\mathrm{intra}} = 120.8\,\mathrm{ns}$, $t_{\mathrm{cross}} = 652.8\,\mathrm{ns}$, $L^{\star} = 2.20$, $R^2 = 0.98$.



Same operator and byte count;
only the row-to-core map changes.

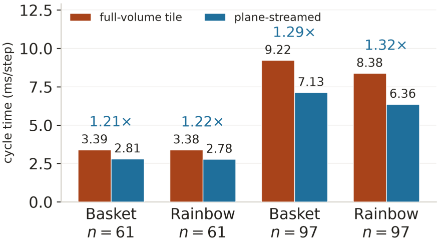
schedule	Rows per core	Speedup
`block`	contiguous band	**36.7**×
`cyclic`	same bytes, dealt out	7.0×
`rotate`	contiguous, reshuffled	6.6×

the OpenMP `schedule` clause

At the knee the implied 176 GB/s exceeds puffin's ∼95 GB/s DRAM roof, so it comes from L3. **Contiguity, not volume, keeps a slice resident.**

On one NUMA node no reduction costs more than a core-complex hop, so synchronization is never dominant: the lever here is **access order**, not communication avoidance.

Appendix: the kernel schedule, not the tile



One V100. The x–y tile is fixed at 16×8; only the order in which a block walks z changes.

Nsight counters, full-volume → plane-streamed

	n=61	n=97
thread blocks launched	2,048 → 256	8,125 → 637
executed instructions	2.9× fewer	3.4× fewer
global-load sectors	2.6–4.2× fewer	

Each plane is fetched from global memory *once* and reused down the segment.

This does not replace *s*-step; it composes with it.

FP64 footprint	n=61	n=337
unknowns N	2.27×10^5	3.83×10^7
one state vector (GB)	1.8×10^{-3}	3.1×10^{-1}
Krylov basis, m=39 (GB)	7.1×10^{-2}	1.19×10^1
share of a 16 GB V100	0.4%	**75%**

$n = 337$ is the largest odd grid common to every topology, payoff and width after a 10% memory reserve. The binding arm is the *one-GPU*, s=4 case, which is exactly why four GPUs start to earn their place above it.

At that grid the predeclared numerical gate converges with maximum $m = 32$ and residual 9.85×10^{-9}, but the boundary-state gate fails at 6.40×10^{-5}. Rescaling the augmentation by $\eta = 2^{-36}$ repairs it to 4.41×10^{-13}; the original stop remains the reported result.